

Close

PIYUSH MODANI  
ASSIGNMENT 11  
ROLL NO 19  
CSAI-D 12410521

PIYUSH MODANI ASSIGNMENT 11 ROLL NO 19 CSAI-D 12410521

Add blockquote

> Add blockquote

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

import kagglehub
```

```
path = kagglehub.dataset_download("mohansacharya/graduate-admissions")

files = os.listdir(path)
csv_file = [f for f in files if f.endswith('.csv')][0]

df = pd.read_csv(os.path.join(path, csv_file))

# Clean column names
df.columns = df.columns.str.strip()

print(df.head())
print(df.columns)
```

Using Colab cache for faster access to the 'graduate-admissions' dataset.

|   | Serial No. | GRE Score | TOEFL Score | University Rating | SOP | LOR | CGPA | \ |
|---|------------|-----------|-------------|-------------------|-----|-----|------|---|
| 0 | 1          | 337       | 118         | 4                 | 4.5 | 4.5 | 9.65 |   |
| 1 | 2          | 324       | 107         | 4                 | 4.0 | 4.5 | 8.87 |   |
| 2 | 3          | 316       | 104         | 3                 | 3.0 | 3.5 | 8.00 |   |
| 3 | 4          | 322       | 110         | 3                 | 3.5 | 2.5 | 8.67 |   |
| 4 | 5          | 314       | 103         | 2                 | 2.0 | 3.0 | 8.21 |   |

|   | Research | Chance of Admit |
|---|----------|-----------------|
| 0 | 1        | 0.92            |
| 1 | 1        | 0.76            |
| 2 | 1        | 0.72            |
| 3 | 1        | 0.80            |
| 4 | 0        | 0.65            |

```
Index(['Serial No.', 'GRE Score', 'TOEFL Score', 'University Rating', 'SOP',
      'LOR', 'CGPA', 'Research', 'Chance of Admit'],
      dtype='object')
```

### STEP 3: Data Preprocessing

```
# Drop Serial Number if exists
for col in df.columns:
    if "serial" in col.lower():
        df.drop(columns=[col], inplace=True)

# Identify target column (Chance of Admit)
target_col = [col for col in df.columns if "chance" in col.lower()][0]

# Convert to classification (0 or 1)
df['Admitted'] = (df[target_col] > 0.75).astype(int)

# Drop original column
df.drop(columns=[target_col], inplace=True)

# Split features & target
X = df.drop(columns=['Admitted'])
y = df['Admitted']

# Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

The problem is converted into a classification task by converting admission probability into binary values (Admitted / Not Admitted).

#### 4: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

Dataset is split into training (80%) and testing (20%) to evaluate model performance on unseen data.

#### 5: Apply Decision Tree

```
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)
```

DecisionTreeClassifier

DecisionTreeClassifier(random\_state=42)

Decision Tree model is trained using Gini Index to find the best splits in the data.

#### 6: Prediction

```
y_pred = model.predict(X_test)
print("Sample Predictions:", y_pred[:10])
```

Sample Predictions: [0 0 1 1 0 1 0 0 0 1]

Model predicts whether a student will be admitted or not based on input features.

#### 7: Model Evaluation

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.9375

Confusion Matrix:

```
[[44  4]
 [ 1 31]]
```

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.98      | 0.92   | 0.95     | 48      |
| 1            | 0.89      | 0.97   | 0.93     | 32      |
| accuracy     |           |        | 0.94     | 80      |
| macro avg    | 0.93      | 0.94   | 0.94     | 80      |
| weighted avg | 0.94      | 0.94   | 0.94     | 80      |

Accuracy shows overall correctness, while confusion matrix and classification report give detailed performance analysis.

#### 8: Visualization (Decision Tree)

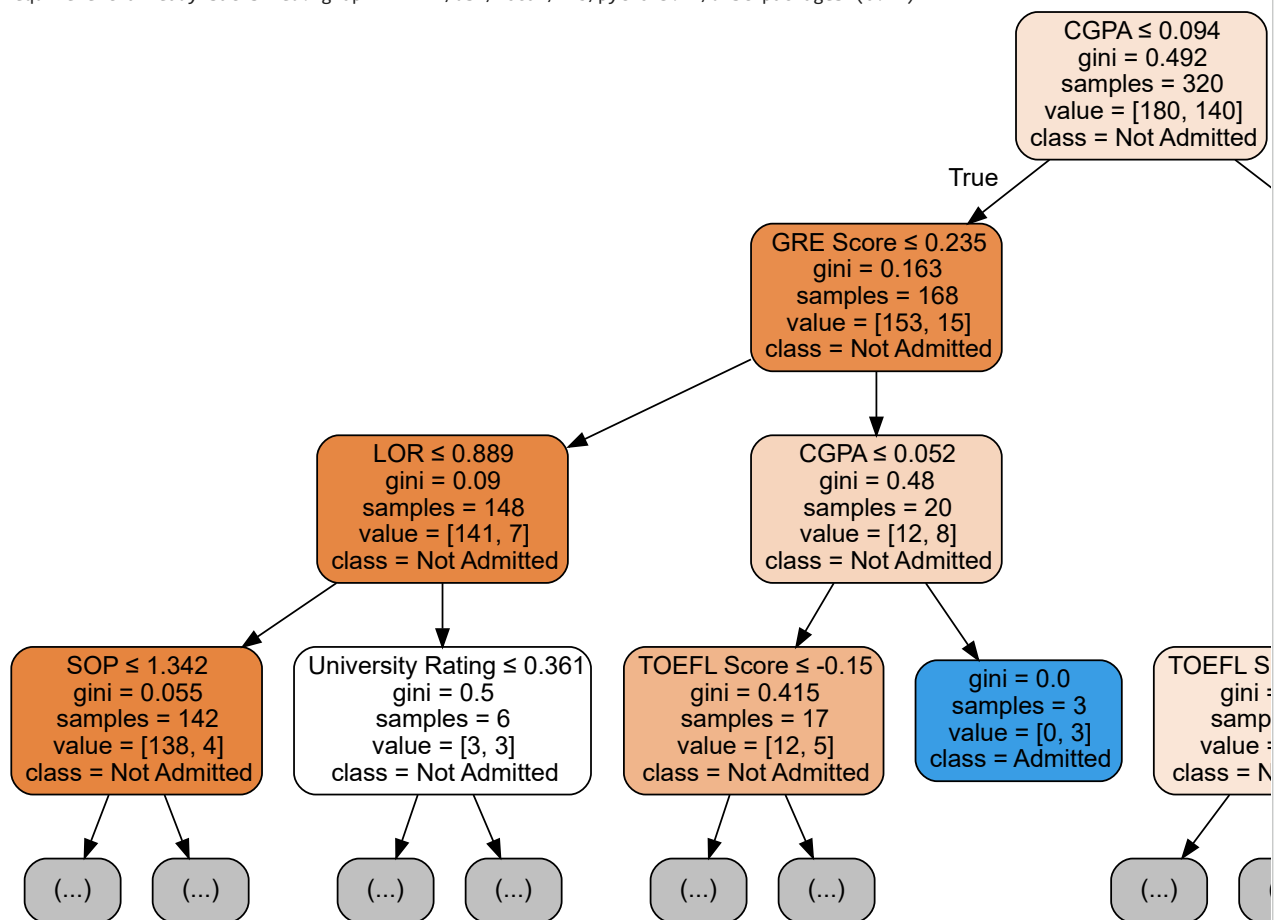
```
# Install graphviz (run once in Colab)
!apt-get install graphviz -y
!pip install graphviz

from sklearn.tree import export_graphviz
import graphviz
```

```
# Export tree
dot_data = export_graphviz(
    model,
    out_file=None,
    feature_names=X.columns,
    class_names=["Not Admitted", "Admitted"],
    filled=True,
    rounded=True,
    special_characters=True,
    max_depth=3 # LIMIT depth for readability
)
```

```
# Display tree
graph = graphviz.Source(dot_data)
graph
```

Reading package lists... Done  
 Building dependency tree... Done  
 Reading state information... Done  
 graphviz is already the newest version (2.42.2-6ubuntu0.1).  
 0 upgraded, 0 newly installed, 0 to remove and 6 not upgraded.  
 Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (0.21)



The tree visualization helps understand how decisions are made based on GRE, CGPA, and other features.

:FINAL INTERPRETATION (WRITE THIS) The Decision Tree classifier successfully predicts student admission chances. Features like GRE Score, CGPA, and TOEFL Score have strong influence. The model achieved good accuracy, indicating reliable predictions. The system can assist counsellors in decision-making.

